ECE 411 Final Report: MP Out Of Order (ERR Implementation)

Group name: TLB Flushers

Group Team members: Akshat Singh (akshat4), Ribhav Sahu (ribhavs2), Atharva Jain (atharva7)

Architecture Implementation: ERR (Explicit Register Renaming)

Introduction & Project Overview:

Modern processors are tasked with achieving high performance while efficiently handling a broad spectrum of computational workloads. The increasing complexity of applications and demand for faster execution has driven the development of sophisticated architectures like out-of-order (OoO) execution. After the implementation of a pipelined processor along with a pipelined cache, we realized the various bottlenecks in that design. It inevitably became clear that out-of-order execution is essential along with various pipelines in that design. In this project, we present the design and implementation of an out-of-order RISC-V processor with Explicit Register Renaming (ERR) and several advanced architectural features aimed at enhancing performance, accuracy, and efficiency.

The design incorporates a robust pipeline with **advanced branch prediction techniques** such as a 2-level branch predictor, Gshare predictor, and a tournament predictor to minimize control hazards and improve instruction fetch accuracy. Additionally, we implemented a **next-line prefetche**r to mitigate latency by preloading potential future cache lines and a **post-commit store buffer** to optimize memory operations post-execution. Furthermore, a **return address stack** ensures accurate handling of function calls and returns, while a detailed **benchmark analysis** evaluates the processor's performance under various workloads.

This project gave us the opportunity to work and tackle industry-level problems that RTL designers face while making a processor. It provided us with a comprehensive understanding of computer architecture principles, including pipeline hazards, branch prediction, memory hierarchy, and execution efficiency. By completing this design, we have not only gained insights into the intricacies of advanced processor features but also developed skills critical for real-world processor design and optimization.

Organization of the Report

1. Architectural Overview:
    1. Overview of the out-of-order RISC-V design and the role of Explicit Register Renaming (ERR).
    2. High-level block diagram and pipeline architecture.
    3. Branch Prediction
    4. Division of work across different Checkpoints, and how the processor looked like after each Checkpoint
2. Description of the implemented branch predictors:

    1. 2-Level Branch Predictor
    2. Gshare Predictor
    3. Tournament Predictor

3. Functionality and design of the Next-Line Prefetcher.
4. Implementation of the Post-Commit Store Buffer and its effect on memory throughput.
5. A detailed explanation of the Return Address Stack and its role in handling function calls and returns.
6. Benchmark analysis and quantitative assessment of the processor's performance across various workloads.
7. Conclusion and Future Work

Milestones

Checkpoint 1

The first milestone focused on designing and implementing the **fetch stage** of the processor. This stage is responsible for retrieving and queuing instructions from memory/cache for decoding. This checkpoint established the foundation for subsequent stages by addressing key challenges such as burst memory responses, proper instruction queueing, and correct communication between components.

The major components implemented in this phase were:

- **Instruction Cache (from mp_cache):** The instruction cache from mp_cache was integrated into the fetch stage to reduce memory access latency. It provides fast access to frequently used instructions, minimizing the need for DRAM accesses.
- **Cacheline Adapter**: The cacheline adapter was designed to interface with the DRAM model. Since the DRAM responds with bursts of four 64-bit words (256 bits), the cacheline adapter:
- **Deserializes Bursts**: Combines the burst responses into a single 256-bit cache line.
- **State Machine**: Utilizes a finite state machine (FSM) with states for idle, bursting, and done to manage the data flow.
- **Accumulator Logic**: Aggregates the burst responses and ensures they are ready for the instruction cache to use.
- **Instruction FIFO Queue:** The FIFO queue buffers fetched instructions to decouple the fetch and decode stages. It uses a circular buffer structure with:
- **Head and Tail Pointers**: Manage enqueue (write) and dequeue (read) operations.
- **Full and Empty Flags**: Indicate whether the queue is ready for new instructions or if it is empty.
- **Overflow Handling**: Tracks when the tail pointer laps the head

- **Integration of the Fetch Stage with the DRAM Model:** The fetch stage handles the retrieval of instructions from memory or the instruction cache. Key features include:
    - Communication with the DRAM model through a cacheline adapter to handle bursts of data.
    - Use of an instruction FIFO queue to buffer fetched instructions for decoding.
    - Integration with the instruction cache to optimize memory access latency.

Testing and Verification:

1. **Cacheline Adapter:**

a. Tested after connecting this to fetch stage

b. Verified correctness by checking that the accumulated data matches the burst data from memory.

2. **Instruction FIFO Queue:**

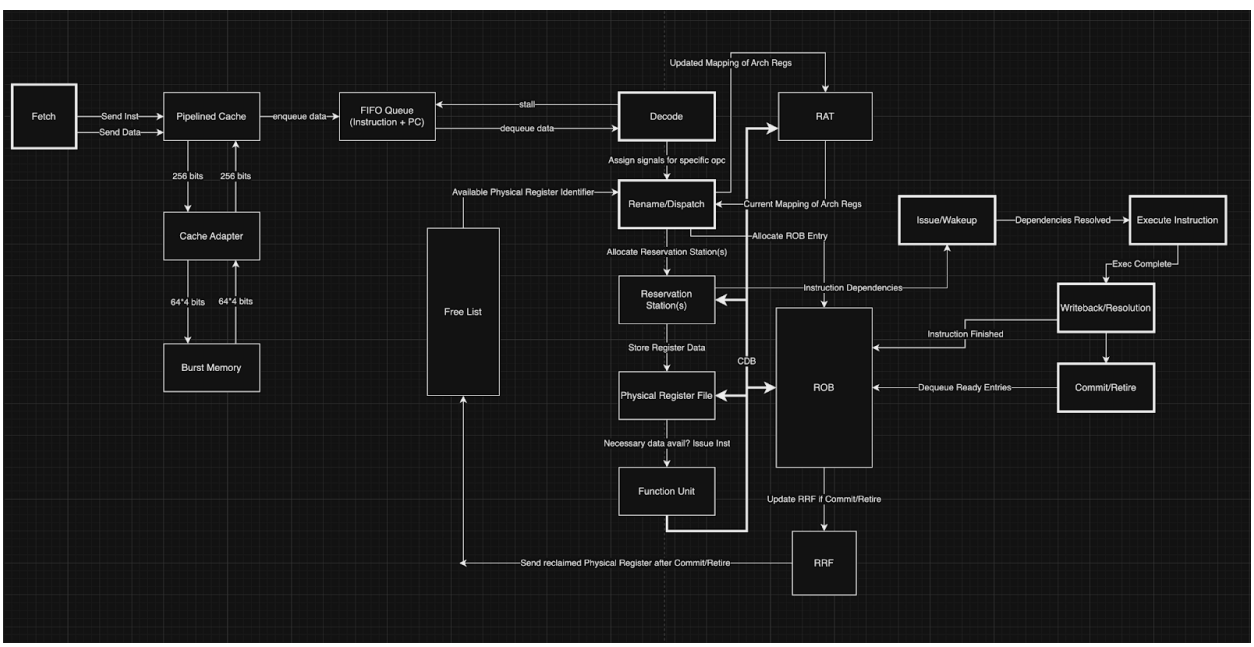
a. Tested with synthetic enqueue and dequeue patterns to ensure proper handling of boundary conditions (e.g., full/empty flags and overflow detection).

b. Verified that the queue correctly stores and retrieves instructions in order with sample programs.

3. **Fetch Stage Integration:**

a. Tested with sample programs, where the fetch stage would fetch instructions and just store them in the instruction FIFO queue.

By the end of this checkpoint, we were also required to design a high-level block diagram of the complete OoO datapath. The following is our initial diagram at the end of checkpoint 1.



Checkpoint 2

The second checkpoint focused on enabling the processor to execute all immediate and register instructions from the RV32I specification, as well as integrating the RISC-V M extension to support multiplication and division instructions. This stage built upon the foundations established in the first checkpoint, incorporating advanced functionality and improving the architectural robustness of the processor. We added all the other stages as seen from our architecture design picture above. This involved designing the ROB, RRF, Free list, reservation stations, RAT, Functional Unit and last but not least, the write back and commit logic to free all the resources that may have been used before. After designing all these, we also had to ensure that all these components were integrated with the components from the first CP, to ensure everything worked together.

Major Components and Implementations:

Immediate and Register Instruction Support: Implemented the full suite of RV32I instructions, ensuring the processor could handle all immediate and register operations as specified in the ISA. Enhanced the Out-of-Order (OoO) execution pipeline to effectively execute these instructions, further developing the fetch, decode, and execution stages.

RISC-V M Extension Integration: Integrated Synopsys multiplier and divider pipelined IPs into the processor for executing RV32M multiplication (MUL) and division/remainder (DIV/REM) instructions. Added specific logic to handle edge cases, such as division by zero, as defined in the RISC-V ISA specification.

Toolflow and Linter Compliance: Introduced Verilator, a cycle-level simulator, to the development toolchain for faster and more efficient testing. This tool provided additional validation for the design and aided in catching potential errors. Ensured compliance with Spyglass and Verilator linters to maintain high coding standards and reliability. Further we were able to make our own test cases to ensure testing alongside changing the random testbench to further test for more edge cases.

Verification and Testing: Modified the mp_verif random testbench to achieve comprehensive coverage of all instructions required in this checkpoint. Demonstrated Out-of-Order execution using the ooo_test.s program to validate the processor's capability to dynamically reorder instructions for improved performance.

Register Alias Table (RAT): The RAT module maps architectural registers to physical registers, ensuring efficient operand management during instruction decoding. It dynamically updates mappings as instructions are dispatched and executed. Operand readiness is determined through interactions with the CDB, allowing for bypassing updated values. Flush operations reset mappings to a consistent state using ROB-provided data. The RAT ensures efficient dependency resolution and operand availability, playing a crucial role in maintaining correct execution and pipeline flow in the ERR processor.

Free List: The Free List manages the allocation and deallocation of physical registers. Implemented as a circular buffer, it tracks available registers using head and tail pointers, with overflow bits managing wrap-around scenarios. Flush events reset the list to ensure system consistency. Enqueue operations add freed registers, while dequeue operations allocate new ones to instructions. This module is essential for managing register usage efficiently, avoiding allocation conflicts, and maintaining optimal performance in the ERR processor.

Reorder Buffer (ROB): The ROB maintains program order by tracking instructions from dispatch to commit. It ensures updates to architectural states occur only after successful execution, preserving program correctness. Features include handling branch mispredictions via flush mechanisms and interacting with branch predictors to ensure correct outcomes. The ROB integrates with functional units to resolve dependencies, tracks operand readiness, and commits results in order, playing a central role in the processor's out-of-order execution and exception handling.

Reservation Station (RS) : The reservation station buffers instructions until operands are ready for execution. It tracks dependencies and dispatches instructions to functional units when conditions are met. The RS dynamically updates operand availability through the CDB, enabling efficient execution in out-of-order pipelines. It prioritizes ready instructions to optimize throughput and prevent stalling. By managing dependencies and enabling concurrent execution, the RS significantly enhances the performance and efficiency of the ERR processor.

Design Highlights: This checkpoint was pivotal in expanding the processor's functionality and laying the groundwork for more advanced features in subsequent checkpoints. By integrating the RV32M instructions and enhancing the OoO pipeline, the processor demonstrated its ability to handle complex workloads efficiently. Additionally, the adoption of Verilator improved testing capabilities, ensuring a robust and reliable implementation.

Checkpoint 3

The last 3rd milestone focussed on implementing and designing the Control and Data Instruction capabilities in our ERR processor. This stage was responsible for adding functionality for Unconditional/ Conditional Jumps, Branch Instructions and Load Store Instructions from the RISC-V 32I ISA. We had to create another instantiation of our 4-Way SA pipelined cache with read/write functionally to support load stores respectively.

Since our processor executes instructions out of order, it causes no issues for arithmetic instructions, but, for this checkpoint, we are implementing Loads and stores which are memory operations. Since memory instructions can have data hazards such as RAW, WAW, WAR etc. we cannot go through the same pipeline as we were doing in earlier checkpoints. Hence, memory instructions do not go through the general pipeline (executed in order) and they have their own reservation station and queue (LSQ).

The major components that were designed in this checkpoint were:

1. **Data Cache (mp_cache):** A read-and-write cache that was used to support the load and store instructions.
2. **Arbiter**: Since our mp_cache mainly sends 1 memory request either for a read or write at a time, we designed this FSM to be able to switch between data cache and instruction cache based on whether we want to service load stores or fetch new instructions respectively.
3. **Reservation Station**: We created a new reservation for processing memory instructions. When a load or store instruction is fetched from the instruction cache and propagated to the

rename/dispatch stage it gets sent to this particular reservation station instead of the ones we used in checkpoint 2.

4. **Address Adder:** This component was used to calculate the address loading or storing which gets passed on to the LSQ for servicing instruction.

5. **Load Store Queue:** LSQ is a FIFO queue designed to service load and store instructions with the following elements.

a. **Head and Tail Pointers**: Manage enqueue (write) and dequeue (read) operations.

b. **Full and Empty Flags**: Indicate whether the LSQ is ready for new instructions or if it is empty.

c. **Overflow Handling**: Tracks when the tail pointer laps the head

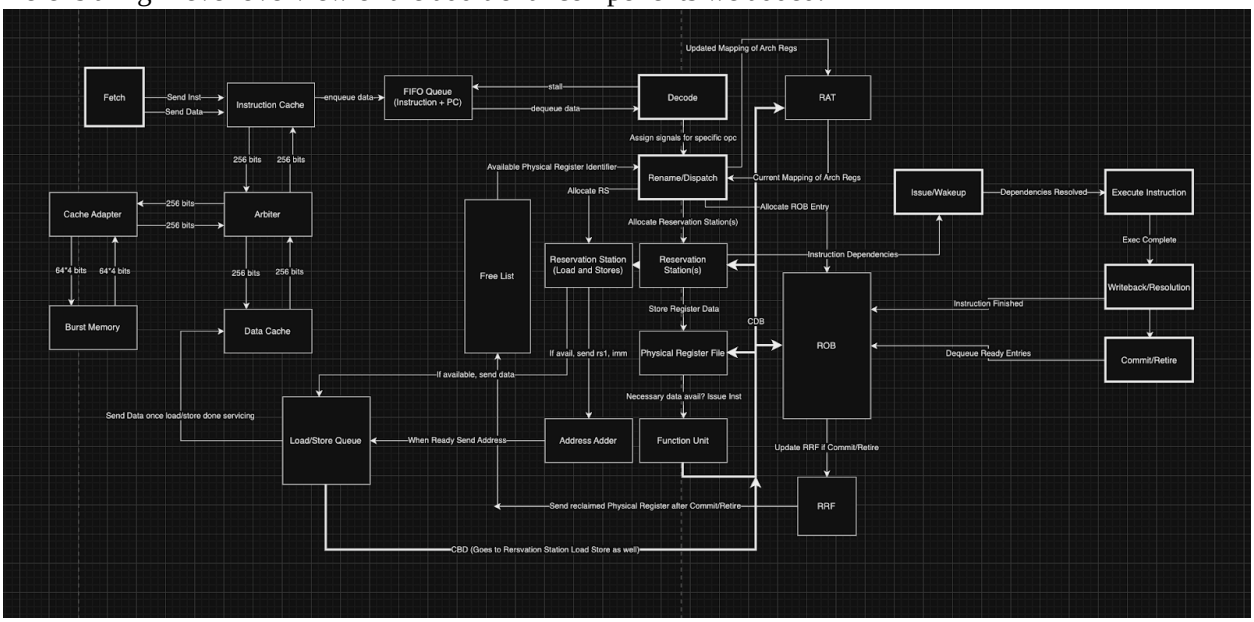
The instructions that are sent into this queue are executed in order since we compare whether an instruction is both at the head of the ROB and LSQ only then do we start calculating the data that has to be either read or written to the cache. This data, once processed, is broadcasted to all through the CDB and gets sent to the data cache.

6. **Static-Taken branch predictor:** We implemented a Static-Taken branch predictor to handle control hazards which are associated with branch and Jump instructions. A static-taken branch predictor always assumes that the control flow decides to jump to the target address. One thing to note is that this strategy works for branch and JAL instructions but does not work for JALR instruction since in a static-taken predictor the target address needs to be calculated before the instruction reaches the execute stage. Since JALR utilized the rs1 register which may be being utilized by some other instruction, we had to move with a static not-taken approach. For tackling mispredictions we were using a **flush** signal which would reset the state of our processor. There are 2 cases.

a. **Branch and JAL:** Flush if the branch taken was a misprediction

b. **JALR**: Flush if the branch not taken was a misprediction.

Here is a high-level overview of the additional components we added:



Advanced Design Features

2 level Branch predictor

Design Description: The Gshare Branch Predictor is a global branch prediction scheme designed to improve prediction accuracy by combining global history and program counter (PC) information. The predictor uses a Global History Register (GHR) to store the outcomes of recently executed branches and performs an XOR operation between the GHR and PC to index the Pattern History Table (PHT). This approach helps capture correlations between branch outcomes across different program locations.

Implementation Details: The module consists of the following key components:

1. State Machine: The state machine transitions between three states: IDLE, READ, and WRITE. It orchestrates the read/write operations and updates to the GHR and PHT.
2. Global History Table (LHT): Tracks the outcomes of the last GHR_LENGTH branches.Updated with the most significant bit of the PHT counter after a prediction is confirmed.
3. Pattern History Table (PHT): Stores 2-bit saturating counters for branch prediction. Indexed using the XOR of the GHR and specific PC bits (GHR_LENGTH + 1:2).
4. Read/Write Operations: The GHR is XORed with the PC bits to compute the index for the PHT. The PHT counter determines the prediction (taken if the counter MSB is set). During a write, we update the PHT counter based on the actual branch outcome. This updates the GHR with the prediction result to maintain a global history.
5. Flush Handling: The module can handle flush scenarios by updating the GHR and PHT appropriately.

Design Trade-offs:

1. Global History vs. Local History: Gshare uses global history, which captures global patterns but might not adapt well to highly localized or specific branch behaviours. Compared to 2-level predictors, Gshare is simpler in design and more suitable for general workloads.
2. Indexing Scheme: The XOR operation between GHR and PC balances between global and local information but could lead to aliasing (different branches mapping to the same PHT entry).
3. Area and Complexity: Gshare requires fewer resources than 2-level predictors as it doesn't need a separate LHT but still improves over static prediction schemes.

Workload Analysis: The Gshare predictor is well-suited for workloads with predictable, repetitive branch patterns influenced by global execution flow. It performs well in loops and large-scale applications where branches follow consistent patterns over time. However, in workloads with highly localized or random branching behavior, Gshare may underperform compared to more sophisticated schemes like the 2-level predictor due to its reliance on global history. The modular nature of the design allows scaling the GHR and PHT sizes to match specific performance and resource constraints, making it versatile across various workloads.\

Performance Impact:

Program	IPC Improvement	Delay Improvement	Power Improvement	PDA Improvement
coremark	8.61%	7.92%	0.69%	21.41%
aes_sha	0.15%	0.15%	-0.70%	-1.60%
cnn	-0.07%	-0.06%	0.20%	-1.33%
compression	2.10%	2.06%	0.28%	5.05%
fft	-0.67%	-0.67%	-0.65%	-4.09%
mergesort	4.00%	3.84%	0.83%	10.62%
raytracing	8.20%	7.59%	0.17%	20.14%

G share Branch Predictor

Design Description: The 2-level Branch Predictor is a dynamic branch prediction scheme designed to improve the accuracy of predicting conditional branches in a pipeline processor. It uses two levels of history: the Local History Table (LHT) and the Pattern History Table (PHT). The LHT stores branch-specific behaviour by maintaining a bit vector (history) for each branch, while the PHT stores counters indexed by the local history to predict the branch outcome. This hierarchical approach allows the predictor to adapt to patterns specific to individual branches, improving accuracy in pipelines with frequent or complex branch instructions.

Implementation Details: The module consists of the following key components:

1. State Machine: Operates in four states: IDLE, READ_LHT, READ_PHT, and WRITE. The state machine handles reading and writing to the LHT and PHT based on signals like read, write, and flush.
2. Local History Table (LHT): Tracks the recent history of each branch. The lht_index is derived from the program counter (PC) bits, and the LHT history is updated with the most recent outcome.
3. Pattern History Table (PHT): Stores 2-bit saturating counters indexed by the LHT history. These counters predict the branch direction.
4. Read/Write Operations: During a read, the LHT is accessed first to retrieve the local history, followed by the PHT to retrieve the branch prediction.During a write, both the LHT and PHT are updated with the latest branch outcomes.
5. Validation Logic: Tracks the validity of LHT and PHT entries and manages updates accordingly.

Design Trade-offs:

1. Accuracy vs. Complexity: The two-level predictor significantly improves prediction accuracy compared to simpler schemes (e.g., static or single-level predictors). However, it adds hardware complexity and requires additional memory resources for LHT and PHT storage.
2. Speed vs. Area: While the design achieves high accuracy, the additional access latency for sequentially reading LHT and PHT may slightly impact performance. Trade-offs were made to balance prediction accuracy and timing.
3. Default Behavior: If LHT or PHT entries are invalid, default values (e.g., strong taken predictions) are used. This ensures the predictor can function even when history is unavailable, though this might introduce occasional mispredictions.

Workload Analysis: The predictor adapts well to workloads with branch-heavy execution patterns. By learning branch-specific behaviours, it excels in scenarios with loops or predictable branch patterns.

However, in workloads with random or highly dynamic branches, the predictor may exhibit lower accuracy due to the limited size of the LHT and PHT. The modular and parameterizable nature of the design allows it to be scaled for different workload requirements, making it versatile for varying processor designs and application domains.

Performance Impact

Note: Since we did not have a viable auto grader run with RAS improvement these are the only programs that can be run locally so we will be testing these and comparing them to the cp3 + branch predictor + next line prefetched + pcsb commit.

Program	IPC Improvement	Delay Improvement
coremark	1.23%	1.22%
aes_sha	7.62%	8.26%
compression	0%	0%
fft	1.9%	1.94%
mergesort	0.38%	0.37%

Tournament Predictor

Design Description: The Tournament Branch Predictor combines the strengths of two distinct branch predictors—Gshare and 2-Level—by dynamically selecting the most accurate predictor for each branch. It uses a Chooser Prediction History Table (CPHT) to track the performance of the predictors and determine which one to trust for a given branch. The tournament predictor provides enhanced accuracy across a range of workloads by leveraging the global history sensitivity of Gshare and the local branch history specialization of the 2-level predictor.

Implementation Details: The tournament predictor utilizes a Chooser Prediction History Table (CPHT) to dynamically determine the most suitable predictor for each branch. The CPHT consists of 2-bit saturating counters indexed by the program counter (PC), where the counter's value indicates the preference for either the Gshare or 2-Level predictor. Specifically, 00 strongly favors Gshare, 11 strongly favors 2-Level, and intermediate values allow for adaptability. During prediction, the CPHT's most significant bit (MSB) is used to select the output from either the Gshare or 2-Level predictor. The correctness of these predictions is evaluated against the actual branch outcome, and the CPHT counter is updated accordingly to favor the more accurate predictor. Ultimately, the tournament predictor provides a final branch prediction (final_prediction) and indicates which predictor was used (branch_used), enabling a dynamic and robust decision-making process to improve accuracy across diverse workloads.

Design Trade-offs:

1. Accuracy vs. Complexity: The tournament predictor is more accurate than standalone Gshare or 2-Level predictors, especially on mixed workloads, but comes at the cost of additional complexity. Managing CPHT requires more area and logic compared to individual predictors.
2. Chooser Table Size: A larger CPHT can improve prediction accuracy by reducing aliasing but increases resource usage and lookup delay.
3. Latency: Combining predictions from both predictors introduces a slight delay due to the CPHT lookup and decision-making process.
4. Branch Misprediction Penalty: While the tournament predictor reduces misprediction rates, incorrect chooser table updates during mispredictions may temporarily degrade performance.

Workload Analysis: The tournament predictor excels in workloads with diverse branch behaviour, where a single predictor cannot consistently perform well. For example:

1. Loops and global patterns benefit from Gshare.
2. Highly localized or irregular patterns are better handled by the 2-level predictor.
3. The dynamic selection mechanism ensures robust performance across a wide variety of applications, making it well-suited for modern processors handling mixed workloads such as web servers, databases, and gaming systems. However, in highly predictable workloads, simpler predictors may achieve comparable accuracy with less complexity.

Performance Impact

Note: Since we did not have a viable auto grader run with RAS improvement these are the only programs that can be run locally so we will be testing these and comparing them to the cp3 + branch predictor + next line prefetched + pcsb commit.

Program	**IPC Improvement**	**Delay Improvement**
coremark	1.23%	1.22%
aes_sha	7.62%	8.26%
compression	0%	0%
fft	1.9%	1.94%
mergesort	0.38%	0.37%

Next Line Prefetcher

Design Description: The Next-Line Prefetcher is an advanced feature to improve the instruction cache efficiency. The core idea behind this feature is to prefetch the cache line immediately following the one currently being fetched by the processor, under the assumption that sequential instruction execution is common in many workloads.

The prefetcher operates alongside the instruction cache. When the fetch stage encounters a cache miss, the prefetcher first checks if the requested instruction has already been pre-fetched. If the instruction is found, it is immediately provided to the instruction cache, avoiding costly memory access. If the instruction is not pre-fetched, the prefetcher fetches the missing instruction from memory, updates the cache, and simultaneously prefetches the next cache line. The prefetched data is stored in a small register array, ready for future access.

Implementation Details: The design includes the following components:

1. **State Machine:** The prefetcher operates using a finite state machine with 4 states:

a. **IDLE:** The prefetcher awaits a request from the instruction cache.

b. **MISS:** The prefetcher initiates a memory access to retrieve the requested instruction.

c. **READ:** The prefetcher provides the requested instruction from its prefetch buffer to the cache, and initiates retrieving the next cache line (transitions to PRE_FETCH stage) since the current one has been read by the cache and no longer needs to be stored by the prefetcher.

d. **PRE_FETCH:** The prefetcher retrieves the next cache line into its buffer for potential future use**.**

2. **Buffer:** A single-line prefetch buffer stores the prefetched cache line along with its associated address. The buffer also uses a valid bit to indicate whether the data is ready for use.
3. **Address Logic:** The prefetcher calculates the address of the next cache line using the current instruction's address. The current implementation always assumes sequential memory accesses.

Design Trade-offs:

- **Benefits:**
    - Reduces instruction fetch latency for sequential workloads by prefetching cache lines likely to be accessed next.
    - Improves overall processor throughput by minimizing stall cycles caused by instruction cache misses.
- **Drawbacks:**
    - Adding a buffer, FSM logic, and associated control signals increases the hardware complexity and area requirements.
    - Prefetching data that is not subsequently used, especially in non-sequential or branch-heavy workloads, can lead to inefficient use of memory bandwidth

Workload Analysis: The Next-Line Prefetcher is most effective for workloads with a high degree of sequential instruction execution, such as loop-intensive or linear control flow programs. In these cases, the prefetched instructions align well with the processor's needs, leading to reduced fetch delays and improved performance. Conversely, the prefetcher may perform poorly for workloads with frequent branches or irregular control flow, as these patterns can result in unused prefetched data. This not only wastes memory bandwidth but can also lead to unnecessary memory accesses, potentially degrading overall performance in such scenarios.

Performance Impact:

Note: The following improvement is compared against base cp3 + 2-level branch predictor

Program	IPC Improvement	Delay Improvement	Power Improvement	PDA Improvement
coremark	2.68%	2.60%	0.56%	7.97%
aes_sha	5.70%	5.38%	1.60%	16.52%
cnn	0.02%	0.00%	0.96%	0.81%
compression	0.00%	0.00%	0.14%	-0.02%
fft	1.42%	1.39%	0.14%	4.08%
mergesort	0.00%	0.00%	-0.26%	-0.42%
raytracing	0.03%	0.02%	0.52%	0.42%
rsa	-2.23%	-2.28%	1.23%	-5.84%

Post Commit Store Buffer

Design Description: The Post Commit Store Buffer (PCSB) is an advanced feature designed to decouple store operations from the data cache (D-cache) and the Reorder Buffer (ROB), reducing pipeline stalls and improving overall performance. In the base design, stores are executed in program order and must wait for a cache hit before they can commit. This behavior can cause the ROB as well as non-dependent loads to stall, delaying the completion of other instructions.

The PCSB solves this issue by acting as an intermediary buffer where stores are written upon commitment. The buffer holds stores until they are serviced by the D-cache. Additionally, the PCSB

serves as an L0 cache for loads, ensuring that loads can access the most recent data before it reaches the D-cache.

Implementation Details: The PCSB operates as a circular buffer with the following key components:

1. **Queue structure:** Stores are enqueued into the PCSB upon commitment in the ROB and dequeued when they are ready to be serviced.
2. **Forwarding Logic:** When a load's address matches a store in the PCSB:
    1. If the load's rmask aligns with the store's wmask, the data is forwarded directly to the load.
    2. If there is a partial mask conflict, the store must be serviced first to ensure correctness.
3. **Store Servicing:** Stores are written to the D-cache under any of the three conditions:
    1. The PCSB is full, and a new store from the Load/Store Queue (LSQ) needs to be enqueued.
    2. The LSQ is empty, allowing stores to be serviced without interfering with load operations.
    3. A mask conflict arises between a load in the LSQ and a store in the PCSB.
4. **Arbiter for Read Memory Access:**
    1. Since for loads, the data can originate from either PCSB (if earlier stores are writing to the same address) or from the D-cache (if no store in the PCSB is writing to the same address as the load is reading from) an arbiter prioritizes and resolves conflicts.
    2. The arbiter determines whether to forward data from the PCSB or issue a load request to the D-cache from the LSQ.

Design Trade-offs:

- **Benefits:**
    - Reduces stalls in the ROB caused by unresolved stores.
    - Improves pipeline throughput by allowing stores to commit even when the D-cache is busy.
    - Provides faster access for loads through forwarding from the PCSB
- **Drawbacks:**
    - Adds hardware complexity with the implementation of the PCSB and the arbiter.

- Mask conflicts add additional execution overhead as stores must be serviced before loads can proceed.

Workload Analysis: The PCSB is particularly beneficial for workloads with:

- Frequent store operations that would otherwise cause stalls in a tightly coupled ROB and D-cache design.
- Loads that rely on recently committed store values, enabling efficient data forwarding.

However, the PCSB may perform less optimally in:

- Workloads with a high rate of mask conflicts, as these introduce additional latency.
- Scenarios where the buffer depth is insufficient to handle a burst of store operations.

Performance Impact:

Note: The following improvement is compared against base cp3 + 2-level branch predictor + next-line prefetcher.

Program	IPC Improvement	Delay Improvement	Power Improvement	PDA Improvement
coremark	-10.01%	-10.01%	0.21%	-27.41%
aes_sha	-10.89%	-10.88%	-0.2%	-29.78%
cnn	-11.51%	-11.53%	-0.03%	-31.19%
compression	-10.14%	-10.15%	0.2%	-27.7%
fft	-4.5%	-4.48%	-0.23%	-13.63%
mergesort	-5.9%	-5.91%	-0.05%	-17.27%
raytracing	-7.44%	-7.45%	0.19%	-21.05%
rsa	-2.6%	-2.58%	-0.27%	-8.36%

Return Address Stack

Design Description: This specialised hardware feature is used to optimise the handling of function calls and returns, especially for JAL and JALR instructions. When a JAL instruction is received by the FETCH stage it pushes the return address (PC + 4) onto the RAS. When a JALR return instruction is received by

the FETCH stage, the processor can pop from the RAS the predicted address instead of resolving the actual address during the EXECUTE stage.

Implementation Details: The design includes the following components

1. **Stack Pointer -** The stack pointer is the location within the stack where the return address is pushed and/or popped from. The stack pointer updates to the next value if something gets pushed onto the stack.
2. **Stack -** The stack contains 8 entries with 32-bit values which hold the return addresses of any subsequent JAL or JALR instructions.
3. **Pop_en -** This signal is used to indicate when a return address needs to be popped from the stack.
4. **Push_en -** This signal is used to indicate when a return address needs to be pushed on the stack.

Design Trade-offs:

- **Benefits**
    - **Performance**: Although the RAS improves the performance of the processor, by reducing the amount of stalls that are needed in case of mispredicted return instructions and increasing the accuracy of the predicted return addresses.
    - **Accuracy**: Accurate return prediction reduces pipeline stalls, saving power by avoiding unnecessary instruction fetch and decoding stages.
- **Tradeoffs**
    - **Area**: Since our RAS depth is 8 entries and each entry is storing 32 bits of data whose values we have to save in the stack, we require additional area and memory cells to maintain this structure. Also since we are using RAS to calculate pc_next early in our processor, we have to save it in our decode_packet struct which increases the area as well.
    - **Power Consumption**: Maintaining the RAS stack (pushing/popping entries) induces power costs and the control logic for RAS adds to the dynamic power consumption.

Workload Analysis: RAS is good in terms of improving performance when it comes to recursive functions involving repeated calls with predictable return addresses. The RAS stores these addresses, ensuring accurate return address prediction and avoiding pipeline stalls.

Conversely, If a workload has few or no function calls and returns, the RAS remains idle and provides no performance benefits and if there are irregular patterns in call-return behaviour then the RAS number of mispredictions increases or stack data may be corrupted.

Performance Impact

Note: Since we did not have a viable auto grader run with RAS improvement these are the only programs that can be run locally so we will be testing these and comparing them to the cp3 + branch predictor + next line prefetched + pcsb commit.

Program	IPC Improvement	Delay Improvement
coremark	1.23%	1.22%
aes_sha	7.62%	8.26%
compression	0%	0%
fft	1.9%	1.94%
mergesort	0.38%	0.37%

Benchmark Analysis

Design Description: To gain insights into the behaviour of benchmark test cases on the out-of-order RISC-V processor, an analysis was conducted to understand the distribution of instruction types, branch behaviour, and dependencies between loads and stores. This information helps to identify bottlenecks, optimize pipeline performance, and refine processor features.

Implementation Details: Using Python, a script was developed to parse log files generated by Spike, a RISC-V simulator, and compute the following statistics:

1. **Instruction Type Distribution:** Counts and percentages of loads, stores, branches, and arithmetic instructions.
2. **Branch Behavior:** Percentage of branches taken versus not taken
3. **Load/Store Dependencies:** Average distance between dependent loads and stores

The script uses regex to extract relevant fields from the log file, tracks dependencies based on memory addresses and calculates various metrics. Key aspects of the implementation include:

- **Branch Instruction Identification**: Branch instructions were identified based on their opcode (0b1100011) and funct3 values corresponding to branch types (beq, bne, blt, etc.). The script determines whether a branch was taken by comparing the program counter (PC) of the branch instruction with the PC of the next instruction.
- **Dependency Analysis**: Dependencies between loads and stores were tracked using a hash map. The map recorded the last PC where each memory address was accessed, enabling the calculation of the distance between dependent instructions.

Key Results:

Program	Arithmetic Inst Distribution: Percentage (Count)	Load Inst Distribution: Percentage (Count)	Store Inst Distribution: Percentage (Count)	Branch Inst Distribution: Percentage (Count)	Branch Taken Distribution: Percentage (Count)	Branch Not-Taken Distribution: Percentage (Count)	Average Distance Btw Dependent Loads/Stores
coremark	56.81% (173,436)	18.15% (55,398)	5.23% (15,976)	19.81% (60,475)	57.58% (34,821)	42.42% (25,654)	2584.88
compression	66.86% (280,838)	11.01% (46,252)	10.12% (42,509)	12.01% (50,469)	70.26% (35,459)	29.74% (15,010)	16.56
aes_sha	57.87% (376,212)	27.87% (181,197)	13.88% (90,241)	0.37% (2,405)	83.53% (2,009)	16.47% (396)	73.31
fft	68.90% (356,308)	13.49% (69,782)	14.93% (77,206)	2.67% (13,825)	98.61% (13,633)	1.39% (192)	15.47
mergesort	55.72% (264,703)	14.97% (71,118)	11.63% (55,269)	17.67% (83,961)	48.56% (40,775)	51.44% (43,186)	2.97

Conclusion:

This project represents the culmination of a collaborative effort to design and implement an out-of-order RISC-V processor featuring Explicit Register Renaming (ERR). As a team, we successfully integrated advanced architectural features, including multi-level branch predictors, a next-line prefetcher, and dynamic memory operation management. These enhancements collectively improved instruction throughput minimized pipeline stalls, and demonstrated the efficiency of modern out-of-order execution techniques.

The development process was a concerted team effort, with each member contributing to critical components such as the branch predictors, Reorder Buffer (ROB), and Load Store Queue (LSQ). Regular collaboration and iterative problem-solving ensured that all modules were seamlessly integrated, resulting in a cohesive and well-optimized processor design. Benchmark analysis confirmed the processor’s adaptability and performance across varied workloads, validating our design choices.

Beyond technical achievements, this project provided invaluable learning opportunities. It deepened our understanding of computer architecture, particularly in areas such as pipeline hazards, branch prediction, and memory hierarchies. The challenges encountered—ranging from handling mispredictions to

optimizing resource utilization—mirrored real-world complexities faced by industry professionals, enabling us to cultivate both technical and collaborative skills.

This endeavour highlights the importance of teamwork in addressing multifaceted engineering challenges and underscores the value of a methodical, iterative approach to problem-solving. The knowledge and skills gained through this project lay a strong foundation for future work in advanced microarchitecture design and optimization.